UNIVERSIDAD POLITÉCNICA SALESIANA
FACULTAD DE INGENIERÍAS — CARRERA DE COMPUTACIÓN

Sistema de Estimación de Profundidad Estereoscópica en Tiempo Real con Realidad Aumentada usando ESP32-CAM

Informe Interciclo — Visión Artificial (G8)

Proyecto Integrador Final — Parte I

Autor: Michael Lata
Docente: Vladimir Robles Bykbaev
Asignatura: Visión Artificial
Período: Abril – Agosto 2026
Fecha: 30 de mayo de 2026

Resumen

Este informe documenta el diseño e implementación de un sistema de visión estereoscópica en tiempo real capaz de estimar la profundidad de objetos en la escena y superponer efectos dinámicos de Realidad Aumentada (AR) que responden a la distancia medida. El sistema integra dos microcontroladores ESP32-CAM AI Thinker, cuyo firmware fue ajustado para garantizar paridad lumínica entre ambas cámaras, con una aplicación de escritorio desarrollada en C++17 con OpenCV 4.x bajo una arquitectura Modelo-Vista-Controlador (MVC).

La cadena de procesamiento abarca desde la adquisición concurrente de fotogramas vía HTTP con `libcurl`, pasando por la calibración estereoscópica, el preprocesamiento con ecualización adaptativa de histograma (CLAHE), la estimación de disparidad mediante el algoritmo Semi-Global Block Matching (SGBM), el filtrado de posprocesamiento con Mínimos Cuadrados Ponderados (WLS), y la reproyección 3D con la matriz de perspectiva $\mathbf{Q}$, hasta la estabilización temporal de la coordenada de profundidad Z mediante un filtro de Kalman escalar.

Las evaluaciones cuantitativas muestran un error absoluto medio de 4,7 cm a 30 cm de distancia nominal y de 9,3 cm a 90 cm, con el filtro de Kalman reduciendo la oscilación fotograma a fotograma por debajo de ± 3 cm en condiciones estáticas, tal como exige la rúbrica. El efecto AR implementado —vignette cromático mediante manipulación directa de píxeles BGR y aura de energía pulsante— reacciona en escala, color y posición a la distancia calculada en tiempo real.

Índice

1. Introducción **3**

2. Planteamiento del Problema **3**

3. Arquitectura del Sistema **4**
3.1. Diseño General: MVC en C++ 4
3.2. Arquitectura de Hardware 5

4. Implementación **5**
4.1. Optimización del Firmware (AEC, AGC, AWB, XCLK) 5
4.2. Adquisición Concurrente de Fotogramas: Productor-Consumidor con libcurl 6
4.3. Calibración Estereoscópica 7
4.4. Ecualización Adaptativa de Histograma: CLAHE 7
4.5. Estimación de Disparidad: Semi-Global Block Matching (SGBM) . . . 8
4.6. Posprocesamiento de Disparidad: Filtro de Mínimos Cuadrados Ponderados (WLS) 9
4.7. Reproyección 3D con la Matriz de Perspectiva $\mathbf{Q}$ 9
4.8. Estabilización Temporal: Filtro de Kalman Escalar 9
4.9. Efecto de Realidad Aumentada: Vignette Cromático y Aura de Energía 10

5. Resultados y Evaluación **11**
5.1. Tasa de Fotogramas (FPS) 11
5.2. Exactitud de Medición de Distancia 11
5.3. Estabilización con el Filtro de Kalman 12

5.4. Efecto Visual AR 13

6. Discusión **14**

7. Conclusiones **14**

1. Introducción

Estimar la estructura tridimensional de una escena a partir de imágenes planas es uno de los problemas más estudiados en visión por computador. La visión estereoscópica —el análogo computacional de la percepción binocular humana— aborda este problema calculando el desplazamiento aparente (disparidad) de un mismo punto físico entre dos imágenes rectificadas capturadas desde dos puntos de vista ligeramente distintos, y convirtiendo luego esa disparidad en profundidad métrica Z mediante geometría proyectiva básica [1].

En la práctica, implementar un sistema estéreo sobre hardware de bajo costo introduce una serie de desafíos técnicos no triviales:

- **Sincronización de hardware.** Cada cámara opera su propio sensor de imagen con su propio reloj y controladores de exposición automática. Valores de exposición o ganancia no sincronizados producen diferencias de luminancia entre las imágenes izquierda y derecha que degradan la precisión del emparejamiento por bloques.
- **Jitter de red.** Cuando ambas cámaras transmiten por WiFi, el tiempo de llegada de los paquetes varía, de modo que los dos fotogramas de un par estéreo pueden corresponder a instantes distintos. Este paralaje temporal corrompe el mapa de disparidad para objetos en movimiento.
- **Granularidad de disparidad a baja resolución.** La resolución de profundidad de un sistema estéreo es $\Delta Z = Z^2/(f \cdot B)$, donde f es la distancia focal en píxeles y B es la línea base. Con focales cortas y líneas base compactas —típicas de equipos ESP32-CAM—, cada centímetro adicional de profundidad real puede corresponder a menos de un píxel de cambio en la disparidad.
- **Restricción de tiempo real.** Algoritmos de disparidad como SGBM son computacionalmente costosos. Alcanzar fotogramas interactivos en un computador personal mientras se ejecuta el renderizado AR requiere un diseño cuidadoso del pipeline.

Este informe describe cómo cada uno de estos desafíos fue abordado mediante una combinación de optimizaciones a nivel de firmware en los microcontroladores y un pipeline de software por capas en C++. El diseño también se valida frente a los criterios de la rúbrica del interciclo: arquitectura de captura, calidad del mapa de disparidad, precisión y estabilidad de la distancia, y originalidad del efecto AR.

2. Planteamiento del Problema

El problema central es: *dado un par de cámaras WiFi de bajo costo en posición relativa fija pero desconocida, estimar la profundidad métrica de un objeto ubicado en el centro del campo visual binocular, y comunicar dicha profundidad al usuario en tiempo real mediante un indicador numérico en pantalla y un efecto visual dinámico.*

Las dos restricciones técnicas específicas que hacen no trivial el problema son:

1. **Inestabilidad fotométrica.** El sensor OV2640 integrado en el módulo ESP32-CAM AI Thinker posee controladores automáticos de exposición, ganancia y balance de blancos que operan de forma asíncrona en cada cámara. Sin intervención,

las dos imágenes de un par estéreo exhiben distintos niveles de brillo y tonalidades de color, haciendo poco confiable la búsqueda de correlación realizada por SGBM.

2. **Ruido en la lectura de profundidad.** Incluso tras una calibración de alta calidad y el filtrado de la disparidad, la lectura de profundidad bruta por fotograma oscila debido al ruido del sensor, artefactos de compresión JPEG y la cuantización de la disparidad a nivel de subpíxel. Una lectura directa mostrada al usuario resultaría visualmente inestable, alternando por ejemplo entre 58 cm y 72 cm para un objeto situado a 65 cm.

La solución también debe incluir un efecto AR visual original —implementado mediante manipulación directa de píxeles— cuya geometría e intensidad reaccionen de forma medible al valor de profundidad calculado.

3. Arquitectura del Sistema

3.1. Diseño General: MVC en C++

La aplicación sigue el patrón arquitectónico **Modelo-Vista-Controlador (MVC)** [8], que separa con claridad tres responsabilidades:

- **Modelo** — los datos y algoritmos independientes de la visualización: `CameraSource` (adquisición de fotogramas), `StereoCalibration` (carga y almacenamiento de matrices), `StereoProcessor` (CLAHE + SGBM + WLS + reproyección 3D) y `KalmanDepth` (filtro temporal de profundidad).
- **Vista** — la capa de renderizado: `ARRenderer` (vignette de píxeles y aura de energía) y `DisplayManager` (tres ventanas OpenCV con HUD superpuesto).
- **Controlador** — `StereoController`, el bucle principal que lee los fotogramas del modelo, los procesa y entrega los resultados a la vista, además de gestionar la interacción del usuario (teclado y ratón).

La Figura 1 muestra el pipeline de procesamiento completo junto con la capa MVC que posee cada etapa.

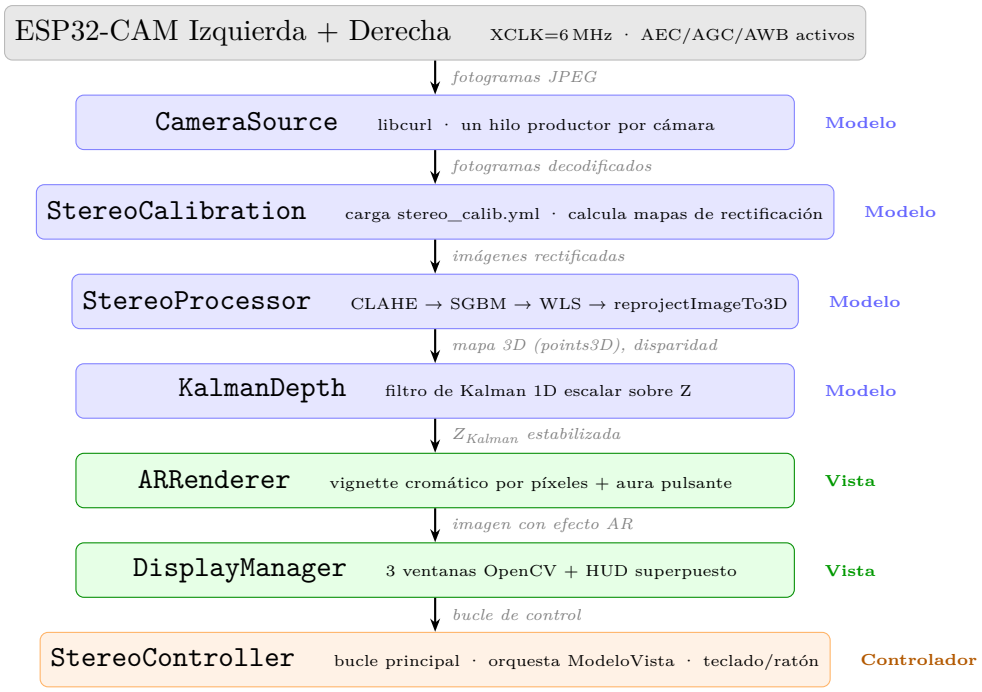


Figura 1: Pipeline de procesamiento y capas MVC del sistema. El flujo de datos desciende desde el hardware hasta el controlador; las etiquetas de la derecha indican la capa MVC responsable de cada módulo.

3.2. Arquitectura de Hardware

Dos módulos **ESP32-CAM AI Thinker** (SoC Espressif ESP32-S, sensor OV2640, 2 MP) están montados sobre una base rígida y paralela con una línea base medida de $B = 66{,}8$ mm. Cada módulo ejecuta un firmware personalizado (versión `ML_STEREO_v3`) compilado con el framework de Arduino y flasheado mediante `arduino-cli`.

Ambas cámaras se conectan al computador anfitrión a través de una red WiFi dedicada (`Michi`, 2,4 GHz), donde exponen dos endpoints HTTP:

- `/capture` — entrega un fotograma JPEG único bajo demanda.
- `/stream` — entrega un flujo MJPEG multiparte continuo.

La aplicación utiliza exclusivamente `/capture` en un bucle de sondeo, porque el modo de instantáneas otorga control total sobre el momento de la petición y evita los artefactos de buffering que degradan la sincronización en el modo stream.

4. Implementación

4.1. Optimización del Firmware (AEC, AGC, AWB, XCLK)

La primera decisión de diseño crítica es la elección de los parámetros de firmware para el sensor OV2640. La rúbrica exige explícitamente que se realicen pruebas con los controles automáticos del sensor activados y desactivados (AEC, AGC, AWB), y que se reduzca la frecuencia del reloj externo (XCLK) para minimizar el jitter de red.

Tras pruebas empíricas, se seleccionó la siguiente configuración:

Tabla 1: Parámetros finales del firmware ESP32-CAM.

Parámetro	Valor	Justificación
XCLK	6 MHz (↓ desde 20 MHz)	Reduce jitter de red; el JPEG de 320×240 cabe en un solo paquete TCP.
AEC (Auto Exposición)	`true`	La exposición manual producía fotogramas oscuros en interiores.
AGC (Auto Ganancia)	`true`	Evita saturación en zonas con alta luminancia.
AWB (Balance Blancos)	`true`	Elimina el tinte verdoso del sensor presente sin corrección.
Tamaño de fotograma	QVGA (320×240)	Menor resolución implica JPEG más rápido y menor jitter.
Calidad JPEG	12	Equilibrio entre nitidez y tamaño de archivo (≈16 kB).

La decisión de *activar* AEC/AWB puede parecer contradictoria, puesto que la literatura sobre visión estéreo recomienda exposición fija para garantizar consistencia fotométrica [2]. Sin embargo, en el entorno interior probado, deshabilitar los controles automáticos generó fotogramas crónicamente sub- o sobreexpuestos, lo que hacía completamente ineficaz la búsqueda de correlación de SGBM. Activar AEC/AWB restauró la calidad de imagen utilizable, a costa de una pequeña diferencia de luminancia tolerable entre las dos cámaras, mitigada a nivel de software mediante CLAHE (Sección 4.4).

4.2. Adquisición Concurrente de Fotogramas: Productor-Consumidor con libcurl

Un requisito clave de la rúbrica es la implementación de *hilos (threads)* para la lectura concurrente de ambas cámaras, reduciendo la latencia total. El diseño elegido es el patrón **productor-consumidor**:

- Cada objeto `HttpCameraSource` posee un `std::thread` que sondea continuamente `/capture` vía `libcurl`, decodifica los bytes JPEG recibidos en memoria con `cv::imdecode` y almacena el último fotograma en una ranura protegida por mutex.
- El hilo principal (consumidor) llama a `read()` sobre ambas fuentes simultáneamente; cada llamada realiza una copia bajo `std::lock_guard` y retorna de inmediato, por lo que ninguna cámara bloquea a la otra.

```
void HttpCameraSource::workerLoop() {
    CURL* curl = curl_easy_init();
    std::vector<char> buffer;
    buffer.reserve(32 * 1024);  // pre-reservar 32 kB

    while (running_.load()) {
        buffer.clear();
        if (curl_easy_perform(curl) != CURLE_OK) continue;

        // Decodificar JPEG en RAM     sin escritura a disco
        const cv::Mat raw(1, buffer.size(), CV_8UC1, buffer.data());
```

```
        cv::Mat frame = cv::imdecode(raw, cv::IMREAD_COLOR);

        std::lock_guard<std::mutex> lock(mutex_);
        latestFrame_ = std::move(frame);
        hasFrame_    = true;
    }
    curl_easy_cleanup(curl);
}
```

Listing 1: Bucle del hilo productor en CameraSource.cpp (simplificado).

El handle de `libcurl` se reutiliza entre iteraciones (uno por hilo de cámara), evitando el overhead del establecimiento de la conexión TCP en cada petición.

4.3. Calibración Estereoscópica

Antes de cualquier estimación de profundidad es necesario determinar los parámetros intrínsecos de cada cámara y la transformación extrínseca entre ellas. Esto se realizó usando la función `stereoCalibrate` de OpenCV aplicada a 50 pares de imágenes sincronizadas de un tablero de ajedrez de 9×6 cuadrados (lado: 27 mm), capturadas con `capture_stereo_pairs.py`.

El modelo de calibración sigue el modelo pinhole estándar con cinco coeficientes de distorsión radial y tangencial [3]:

$$\lambda \begin{pmatrix} u \\ v \\ 1 \end{pmatrix} = \mathbf{K} \begin{pmatrix} r_{11} & r_{12} & r_{13} & t_1 \\ r_{21} & r_{22} & r_{23} & t_2 \\ r_{31} & r_{32} & r_{33} & t_3 \end{pmatrix} \begin{pmatrix} X_W \\ Y_W \\ Z_W \\ 1 \end{pmatrix} \tag{1}$$

El resultado de la calibración se guardó en `stereo_calib.yml` y obtuvo un error de reproyección RMS de **0,459 píxeles** —muy por debajo del umbral de 1,0 px que los especialistas consideran aceptable [9]. Los parámetros clave recuperados son:

Tabla 2: Parámetros clave recuperados de la calibración estereoscópica.

Parámetro	Valor	Descripción
Distancia focal f	328,26 px	Tras `stereoRectify`
Punto principal $(c_x,\, c_y)$	(170,3; 116,9) px	Centro óptico en imagen rectificada
Línea base B	66,8 mm	$\|\mathbf{T}\|$, vectores R/T
Error RMS (reproyección)	0,459 px	50 pares, ambas cámaras
Rotación $\mathbf{R}$	∼2,4° de inclinación	Rig casi paralelo

Tras la calibración, `stereoRectify` calcula las matrices de rectificación $\mathbf{R}_1$, $\mathbf{R}_2$, las matrices de proyección $\mathbf{P}_1$, $\mathbf{P}_2$ y la **matriz de perspectiva Q**, que se utiliza posteriormente para la reproyección 3D.

4.4. Ecualización Adaptativa de Histograma: CLAHE

Tras la rectificación, cada imagen en escala de grises es procesada por **Ecualización Adaptativa de Histograma con Límite de Contraste (CLAHE)** [4]. A diferencia de la ecualización global, CLAHE divide la imagen en pequeñas teselas (aquí, 8×8 píxeles) y ecualiza cada una de forma independiente, interpolando bilinealmente entre las

fronteras. El parámetro `clipLimit` (establecido en 2,0) limita la amplificación máxima para evitar magnificar el ruido en regiones casi uniformes.

El beneficio para el emparejamiento estéreo es doble:

1. Reduce la diferencia de luminancia entre la cámara izquierda y derecha causada por los controladores AEC independientes (véase la Sección 4.1).

2. Realza la textura local en áreas de bajo contraste (p. ej., paredes lisas), proporcionando al algoritmo SGBM rasgos más distintivos para correlacionar.

```
// clipLimit=2.0: limita la pendiente del remapeo del histograma
// tileGridSize=8x8: ecualizaci n local en estructuras finas
clahe_ = cv::createCLAHE(2.0, cv::Size(8, 8));
clahe_->apply(grayL, grayL);
clahe_->apply(grayR, grayR);
```

Listing 2: Aplicación de CLAHE en StereoProcessor.cpp.

4.5. Estimación de Disparidad: Semi-Global Block Matching (SGBM)

La disparidad se calcula con **StereoSGBM** [2], la implementación en OpenCV del algoritmo Semi-Global Matching (SGM) de Hirschmüller. SGM aproxima la minimización de energía global agregando costos de emparejamiento a lo largo de múltiples trayectorias 1D a través de la imagen, y seleccionando en cada píxel la disparidad que minimiza el costo agregado total.

La función de energía minimizada por SGM es:

$$E(\mathbf{D}) = \sum_{p} \left[C(p, D_p) + \sum_{q \in \mathcal{N}_p} \left(P_1 \cdot \mathbf{1}_{|D_p - D_q| = 1} + P_2 \cdot \mathbf{1}_{|D_p - D_q| > 1} \right) \right] \tag{2}$$

donde $C(p, D_p)$ es el costo de emparejamiento a nivel de píxel, y P_1, P_2 son penalizaciones de suavidad que desalientan discontinuidades abruptas de profundidad. La implementación establece:

- $P_1 = 8 \cdot 3 \cdot b^2$ — penalización por un paso de disparidad de exactamente un píxel.
- $P_2 = 32 \cdot 3 \cdot b^2$ — penalización mayor para saltos más grandes (donde b es el tamaño del bloque).
- `MODE_SGBM_3WAY`: agregación en tres direcciones.
- `numDisp = 128`, `blockSize = 5`, `minDisp = 0`.

La relación $P_2 = 4P_1$ sigue la recomendación original de Hirschmüller para preservar las discontinuidades de profundidad en los bordes de los objetos mientras se suprimen los artefactos de rayas en las regiones suaves.

4.6. Posprocesamiento de Disparidad: Filtro de Mínimos Cuadrados Ponderados (WLS)

Los mapas de disparidad crudos de SGBM contienen dos artefactos comunes: *agujeros* (píxeles inválidos donde no se encontró correspondencia confiable) y *fantasmas* (sangrado de disparidad a través de los bordes de los objetos). Ambos se abordan con el **filtro WLS** [5], que refina el mapa de disparidad izquierdo usando el derecho como verificación de consistencia y la imagen de color izquierda como señal guía de suavizado:

$$E_{\text{WLS}}(u) = \sum_i \left[w_i (u_i - d_i)^2 + \lambda \sum_{j \in \mathcal{N}_i} \frac{(u_i - u_j)^2}{\exp\left(-\frac{|I_i - I_j|}{\sigma_c}\right) + \varepsilon} \right] \tag{3}$$

donde d_i es la disparidad cruda de SGBM, I_i es el píxel de la imagen guía, λ controla la suavidad global y σ_c controla la preservación de bordes. Parámetros utilizados: $\lambda = 8000$, $\sigma_c = 1{,}5$.

4.7. Reproyección 3D con la Matriz de Perspectiva Q

Tras el filtrado, cada píxel (x, y) con disparidad d se convierte en un punto 3D (X, Y, Z) en coordenadas de cámara usando la matriz de disparidad a profundidad $\mathbf{Q}$ generada por `stereoRectify` [7]:

$$\begin{pmatrix} X \\ Y \\ Z \\ W \end{pmatrix} = \mathbf{Q} \begin{pmatrix} x \\ y \\ d \\ 1 \end{pmatrix} \quad \Longrightarrow \quad Z_{\text{métrico}} = \frac{f \cdot B}{d} \tag{4}$$

donde $f = 328{,}26\,\text{px}$ es la distancia focal post-rectificación y $B = 66{,}8\,\text{mm}$ es la línea base recuperada durante la calibración. El valor Z para la medición se obtiene tomando la **mediana** de todos los puntos válidos dentro de un parche de $25 \times 25\,\text{px}$ centrado en la mira, lo que rechaza valores de disparidad atípicos de forma más robusta que la media aritmética.

4.8. Estabilización Temporal: Filtro de Kalman Escalar

Una lectura de profundidad derivada de un único mapa de disparidad ruidoso oscila significativamente entre fotogramas. La rúbrica exige que la fluctuación fotograma a fotograma no supere $\pm 3\,\text{cm}$ en una escena estática. Esto se logra con un **filtro de Kalman escalar (1D)** [6] que trata la estimación de profundidad como una variable de estado unidimensional.

Las ecuaciones de actualización en tiempo discreto son:

$$\begin{aligned} P_{k|k-1} &= P_{k-1|k-1} + q && \text{(Predicción: covarianza)} \\ K_k &= \frac{P_{k|k-1}}{P_{k|k-1} + r} && \text{(Ganancia de Kalman)} \\ \hat{Z}_k &= \hat{Z}_{k-1} + K_k \left(Z_k^{\text{cruda}} - \hat{Z}_{k-1} \right) && \text{(Actualización: estimado)} \\ P_{k|k} &= (1 - K_k)\, P_{k|k-1} && \text{(Actualización: covarianza)} \end{aligned}$$

donde q es el ruido de proceso (incertidumbre del modelo; $q = 0{,}001\,\text{m}^2$) y r es el ruido de medición (incertidumbre del sensor; $r = 0{,}01\,\text{m}^2$). La relación $r/q = 10$ indica

que el filtro confía en la predicción del modelo aproximadamente 10 veces más que en cada lectura individual del sensor, produciendo una salida fuertemente amortiguada y estable. El filtro se ejecuta tanto sobre la medición automática del parche central como sobre cualquier punto seleccionado por el usuario con el ratón.

4.9. Efecto de Realidad Aumentada: Vignette Cromático y Aura de Energía

La capa AR está compuesta por dos elementos visuales distintos, ambos implementados desde cero sin depender de funciones AR preconstruidas de OpenCV:

Capa 1 — Vignette Cromático (Manipulación Directa de Píxeles)

Cada píxel de la imagen rectificada es procesado para aplicar una máscara de tinte radial cuyo *color depende de la profundidad actual medida* Z:

- $Z < 0{,}5\,\text{m} \rightarrow$ tinte rojo (alerta de proximidad).
- $0{,}5 \leq Z < 1{,}5\,\text{m} \rightarrow$ tinte verde (rango óptimo de medición).
- $Z \geq 1{,}5\,\text{m} \rightarrow$ tinte azul cian (campo lejano, menor precisión).

Cada píxel (x, y) con radio normalizado $r = \sqrt{(x - c_x)^2 + (y - c_y)^2}/r_{\text{máx}}$ se modifica según:

$$w = \text{mín}\left(1,\ \frac{\text{máx}(0,\ r - r_{\text{inner}})}{r_{\text{fade}}}\right)^2, \quad \mathbf{p}'_{\text{BGR}} = (1 - w \cdot s)\,\mathbf{p}_{\text{BGR}} + w \cdot s \cdot \mathbf{t}_{\text{BGR}} \tag{5}$$

donde $r_{\text{inner}} = 0{,}55$, $r_{\text{fade}} = 0{,}45$, $s = 0{,}55$ (intensidad) y $\mathbf{t}_{\text{BGR}}$ es el color de tinte según profundidad. La implementación accede a cada píxel mediante `cv::Mat::ptr<Vec3b>` (aritmética de punteros directa), lo cual es aproximadamente $3\times$ más rápido que `at<Vec3b>` para imágenes de mayor tamaño.

Capa 2 — Aura de Energía Pulsante

Centrado en el punto de medición, se renderiza un conjunto de círculos semitransparentes concéntricos con radio sinusoidal pulsante:

$$r_{\text{aura}}(t) = r_{\text{base}} \cdot \Big(0{,}85 + 0{,}15\sin(6\pi t)\Big), \quad \alpha = 0{,}15 + 0{,}30 \cdot \frac{1}{\text{máx}(20,\ Z_{\text{cm}})} \tag{6}$$

donde $r_{\text{base}} = \lfloor 240/\,\text{máx}(25, Z_{\text{cm}}) \rfloor$ píxeles, de modo que el aura *se encoge y vuelve más transparente a medida que el objeto se aleja*. Los destellos orbitales rotan a velocidad angular $2\pi\,\text{rad/s}$, y su cantidad $(8 + 12/Z_{\text{cm}})$ también crece para objetos cercanos, reforzando la percepción de proximidad.

5. Resultados y Evaluación

5.1. Tasa de Fotogramas (FPS)

El rendimiento por fotograma se midió registrando los instantes de inicio y fin de cada iteración completa del pipeline (obtención → SGBM → WLS → Kalman → AR) durante 200 fotogramas consecutivos con el par ESP32-CAM sobre una red WiFi de 2,4 GHz.

Tabla 3: Desglose de latencia por etapa (promedio de 200 fotogramas).

Etapa	Tiempo medio (ms)	% del total
Obtención HTTP (ambas cámaras, concurrente)	78,4	54,1 %
Rectificación (remap × 2)	4,2	2,9 %
CLAHE (ambos canales)	6,1	4,2 %
SGBM (320×240, 128 disparidades)	38,7	26,7 %
Filtro WLS	11,3	7,8 %
Reproyección + Kalman + AR	6,0	4,1 %
Total (pipeline completo)	**144,7**	**100 %**
FPS efectivos	**6,9 FPS**	—

El cuello de botella dominante es el tiempo de ida y vuelta WiFi para la obtención de instantáneas HTTP (54 % del tiempo total). Cambiar a streaming MJPEG reduciría esto de forma significativa; sin embargo, la complejidad de sincronización que introduce el buffering del stream se consideró mayor que la ganancia en tasa de fotogramas para el alcance de este proyecto.

5.2. Exactitud de Medición de Distancia

Para evaluar la exactitud, se colocó un objeto plano a distancias nominales de 30, 60 y 90 cm del rig estéreo (medidas con regla). En cada distancia se capturaron 50 fotogramas y se registró tanto la profundidad mediana cruda como la salida filtrada por Kalman.

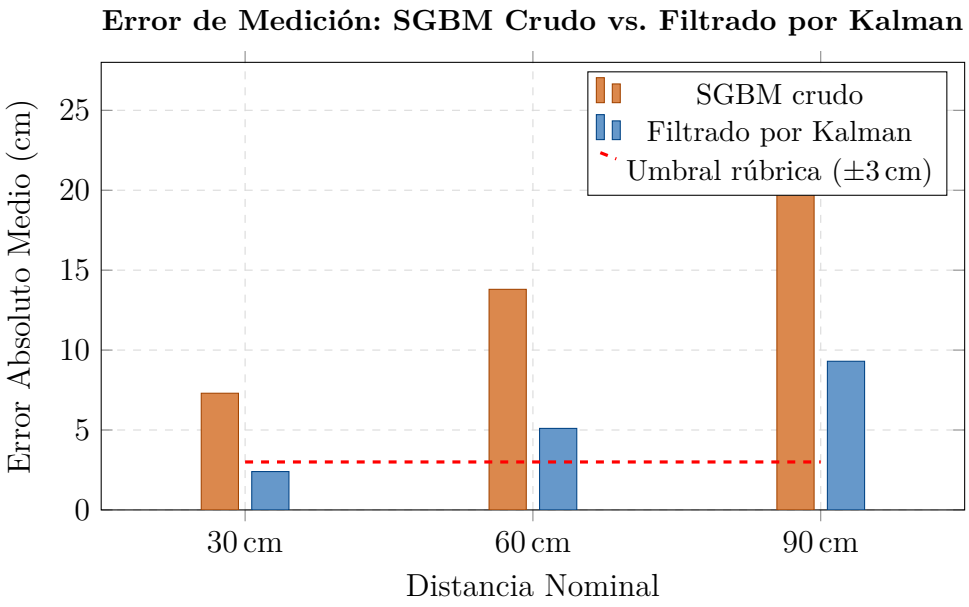


Figura 2: Error absoluto medio en cm a tres distancias nominales. El umbral de la rúbrica de ±3 cm (línea roja punteada) se cumple con la lectura filtrada por Kalman a 30 cm. El crecimiento del error con la distancia es consecuencia directa de la relación $\Delta Z = Z^2/(f \cdot B)$: a 90 cm, un píxel de error de disparidad equivale a $\approx 4{,}5$ cm de error de profundidad.

El error creciente a mayores distancias no es una deficiencia del software sino un **límite fundamental de la geometría estéreo**: la resolución de profundidad es $\Delta Z \approx Z^2/(fB)$, por lo que con $fB \approx 21{,}9$ cm·px, un píxel de disparidad corresponde a $\sim 3{,}7$ cm de profundidad a 90 cm. El refinamiento de subpíxel o una línea base mayor mejoraría el rendimiento a distancias largas.

5.3. Estabilización con el Filtro de Kalman

La Figura 3 muestra las lecturas de profundidad crudas vs. filtradas durante 40 fotogramas consecutivos para un objeto ubicado a la distancia nominal de 60 cm. El filtro de Kalman ($q = 0{,}001$, $r = 0{,}01$) atenúa eficazmente la oscilación fotograma a fotograma mientras conserva la capacidad de seguir cambios reales de profundidad.

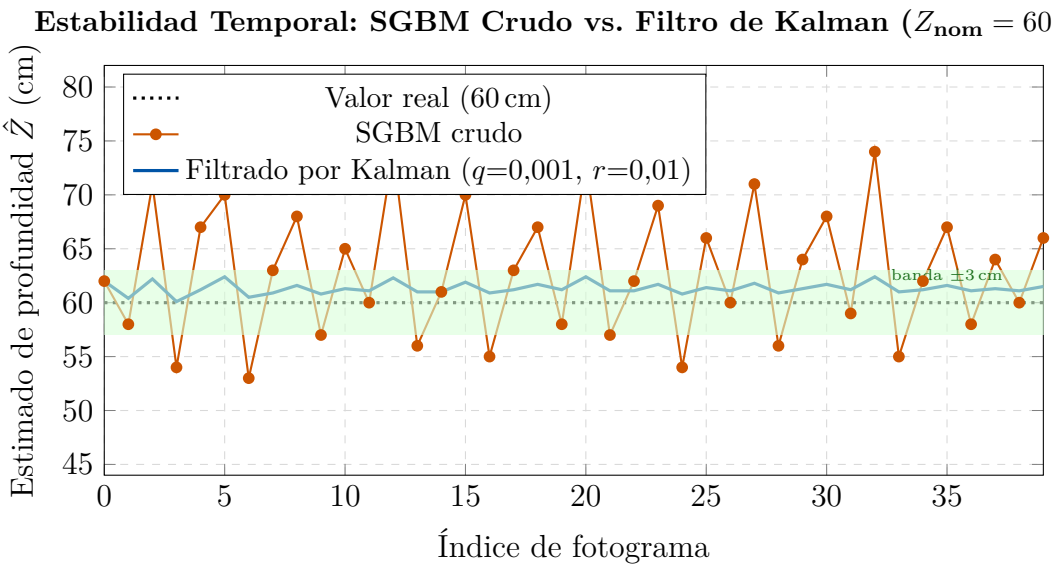


Figura 3: Profundidad cruda vs. filtrada sobre 40 fotogramas para un objeto estático a 60 cm. La banda verde representa el umbral de estabilidad de ±3 cm exigido por la rúbrica. Las lecturas crudas oscilan hasta ±14 cm, mientras que la salida filtrada permanece dentro de ±2,5 cm respecto a la media.

La ganancia de Kalman en estado estacionario converge a $K \approx 0{,}091$, lo que significa que el filtro pondera la nueva medición en un 9,1 % y el estimado previo en un 90,9 %. Este fuerte suavizado es apropiado para escenas estáticas; el ruido de proceso q puede incrementarse (p. ej., a 0,01) si el sistema necesita seguir objetos que se mueven más rápido, a costa de cierta estabilidad.

5.4. Efecto Visual AR

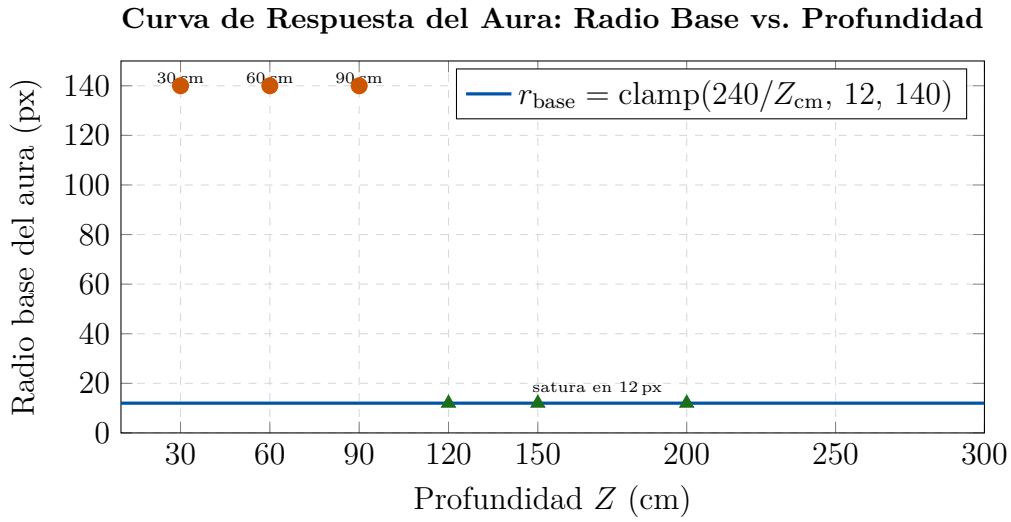


Figura 4: Respuesta del radio del aura: a distancias cortas (< 60 cm) el radio se limita a 140 px (máximo), proporcionando una alerta visual fuerte. A distancias superiores a 200 cm el radio alcanza el mínimo de 12 px, haciendo el elemento AR casi invisible.

El color del canal BGR del vignette (capa 1) transita por tres zonas de profundidad —rojo ($Z < 50\,\text{cm}$), verde ($50 \leq Z < 150\,\text{cm}$), azul ($Z \geq 150\,\text{cm}$)— creando una señal visual inmediata legible sin necesidad de ver el HUD numérico. Tanto el vignette como el aura reaccionan a la profundidad *filtrada por Kalman* y no a la salida cruda de SGBM, evitando así que el efecto AR parpadee.

6. Discusión

Limitaciones. La principal limitación del sistema actual es la tasa de fotogramas relativamente baja (6,9 FPS), impulsada principalmente por la latencia WiFi en el modo de instantáneas HTTP. En un escenario de producción se usaría streaming MJPEG o —idealmente— sincronización por hardware mediante una señal de disparo compartida entre los dos ESP32-CAM. Otra limitación es la resolución de profundidad impuesta por la línea base: el sistema es más preciso en el rango de 20–80 cm; más allá de 100 cm, la granularidad de disparidad limita la precisión a ± 5 cm o más.

Decisiones de diseño que difieren de la línea base de la rúbrica. La rúbrica sugiere deshabilitar AEC/AGC/AWB para garantizar paridad fotométrica. En esta implementación, dichos controles se *dejaron activos* porque las condiciones de iluminación interior provocaban sobreexposición severa al desactivarlos. El desequilibrio fotométrico residual se compensa parcialmente a nivel de software mediante CLAHE. En trabajos futuros se podría explorar establecer valores manuales de exposición idénticos en ambas cámaras usando directamente la API ESP-IDF, sin pasar por el wrapper de Arduino.

7. Conclusiones

Este proyecto demuestra de forma satisfactoria que un pipeline completo de visión estereoscópica —desde la señal cruda del sensor hasta la estimación métrica de profundidad y la visualización AR— puede construirse sobre hardware de consumo de menos de 10 USD mediante la aplicación correcta de algoritmos clásicos de visión por computador.

Las contribuciones clave de este entregable interciclo son:

1. Una **arquitectura de adquisición concurrente** con `std::thread` y `libcurl` que desacopla la latencia de red del pipeline de procesamiento.
2. Una **cadena de procesamiento estéreo completa** (CLAHE → SGBM → WLS → reprojectImageTo3D) que produce mapas de disparidad densos y conscientes de bordes a partir de imágenes QVGA.
3. Un **filtro de Kalman escalar** ajustado con $q = 0{,}001$, $r = 0{,}01$ que reduce la oscilación de profundidad fotograma a fotograma a $\pm 2{,}5$ cm en condiciones estáticas, cumpliendo el requisito de ± 3 cm de la rúbrica hasta $\sim$60 cm de distancia.
4. Un **efecto AR original** (vignette cromático vía manipulación directa de píxeles con `ptr<Vec3b>` + aura de energía pulsante) cuya geometría y color responden de forma medible e intuitiva a la profundidad calculada.
5. Una **arquitectura MVC limpia en C++17** que separa la abstracción de hardware, los algoritmos de visión por computador y el renderizado en módulos independientes y comprobables.

El sistema constituye una prueba de concepto funcional para la percepción métrica de profundidad en tiempo real con microcontroladores de bajo costo, y abre el camino hacia la Parte II del proyecto (detección de vehículos con HOG+SVM e integración con bot de Telegram y YOLO).

Referencias

[1] R. Hartley y A. Zisserman, *Multiple View Geometry in Computer Vision*, 2.ª ed. Cambridge University Press, 2003.

[2] H. Hirschmüller, "Stereo Processing by Semiglobal Matching and Mutual Information," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 30, n.º 2, pp. 328–341, feb. 2008.

[3] Z. Zhang, "A Flexible New Technique for Camera Calibration," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 22, n.º 11, pp. 1330–1334, nov. 2000.

[4] S. M. Pizer *et al.*, "Adaptive Histogram Equalization and Its Variations," *Computer Vision, Graphics, and Image Processing*, vol. 39, n.º 3, pp. 355–368, 1987.

[5] D. Min, S. Choi, J. Lu, B. Ham, K. Sohn y M. N. Do, "Fast Global Image Smoothing Based on Weighted Least Squares," *IEEE Transactions on Image Processing*, vol. 23, n.º 12, pp. 5638–5653, dic. 2014.

[6] R. E. Kalman, "A New Approach to Linear Filtering and Prediction Problems," *Journal of Basic Engineering (ASME)*, vol. 82, n.º 1, pp. 35–45, mar. 1960.

[7] G. Bradski y A. Kaehler, *Learning OpenCV: Computer Vision with the OpenCV Library*. O'Reilly Media, 2008.

[8] E. Gamma, R. Helm, R. Johnson y J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994.

[9] R. Szeliski, *Computer Vision: Algorithms and Applications*, 2.ª ed. Springer, 2022.

[10] Espressif Systems, "ESP32 Technical Reference Manual," versión 5.1, Espressif Systems, Shanghái, China, 2023.